

HAPI Showcase — Requirements Specification & Design Proposal

- **Role:** BA — Imraan Mohammed
 - **Date:** Tuesday, 29th Sep 2026 (Version 7.0)
 - **Status:** Draft (Version 7.0)
 - **Details:** This document consolidates ideas from our signed project proposal and client meeting notes, complimenting them as an extension, providing implementation ready details, including functional and non-functional requirements, in addition to design elements our UX Designer (Max Thum) has already sketched out, and Infrastructure/Hosting Architecture planned by our Dev 1 (Nick Moore). **Revision (v6.0)** considers further client feedback and UX design considerations, including introduction of Hero Banner Images for individual projects used in Featured carousel on Homepage, moving Pending/Approved/Rejected editorial review workflow to Stretch Goal status until Stretch Goal “Student Submission Portal (Student Accounts)” is implemented by Devs, updating Standard Categories from one-to-many to many-to-many relationship with Projects allowing multiple categories to be assigned to individual projects, updating “Showcase” section in Admin Dashboard with support for editing key details associated with them, inclusion of the “Categories” quick link in the menu header for spotlighting specific showcases, confirm webm as the video output format post compression and conversion and estimated processing costs confirmed by Devs, additionally cascading the deletion a Project’s associated media files when the project is deleted, define sorting Date behaviour to be reliant on Showcase Publish Dates and not Project Date Created, include mechanism for selecting Featured Projects with keyboard accessibility for ranking also considered, and update the homepage design layout from UX and Client Feedback to be more “democratically” aligned for addressing client concerns around favouritism amongst student projects.
 - **Related Document:** The signed Client Project Proposal organised by our PM (Nirmal Rajesh) already covers the business context, such as the problem statement, our team roles, target users, stakeholders, proposed solution, success criteria, scope boundaries, 3-sprint roadmap, assumptions and constraints.
 - [Signed Client Project Proposal \(PDF\)](#)
 - **Scope Note:** This scope of this document simply defines the requirements for the project, and any edge cases raised by the client alone. EPICS and User Stories have already been derived from this document as a separate deliverable by our PM (Nirmal Rajesh) during the project planning phase in Sprint 1. As the BA, tracking which Stretch Goals the Devs have implemented and which remain pending shall determine when a Stretch Goal or Optional Feature is promoted to a Core Requirement in this document and future revisions.
-

1. Functional Requirements/Rules

Browsing:

- The homepage shall be publicly accessible without requiring a login, and shall display a browseable grid of featured, published showcase projects.
- Each showcase project shall be displayed as a summary card containing an icon, name, and short description.
 - Clicking or tapping a project card shall navigate the user to that project's dedicated page.
 - Clicking or tapping the "View More" button shall navigate the user to the projects/search page to browse the full list of projects.
- A search bar shall also be available directly on the homepage, allowing users to search without having to navigate to the Projects/Search page.
- The header menu shall provide two distinct quick links, as the first "Applications" shall link to the Projects/Search page for finding individual projects and the second "Categories" representing different Showcases shall link to a dedicated Showcase spotlight page, which shall highlight each Showcase and organise them by Semester and Year.

Individual Project Pages:

- Project Attributes (Fields):

- Icon | Name | Subtitle (Short/Summary Description) | Developer/s | Category | Showcase: Program/Year/Semester (*with sub-categories*) | Date Created (**Optional for information purposes only, for project visibility shall be determined by its associated showcase publish date instead**) | Available On (Supported Devices) : iPhone/iPod, iPad, macOS MacBooks, Apple Watch, Apple TV and/or Vision Pro | Download Link(s) | Contact Us |
- Screenshots Gallery | Video Walkthrough | Description | Hero Banner Image
- **Screenshots Gallery** shall display per supported device (iPhone/iPod, iPad, macOS MacBooks, Vision Pro, Apple Watch, Apple TV) each with a minimum of 4 images, and shall be shown in a carousel/slideshow format. By default, the gallery shall display all screenshots across all supported devices combined in a single carousel, and the user shall then be able to narrow this down to specific device screenshots from there.
- **Video Walkthrough** shall be optional for Demoing the Project/Showcase (**Optional, as students are not required to submit one**):
 - Auto-play feature shall not be implemented, as video playback shall be user-initiated only while still remaining in a muted state by default, reflecting the Apple design principle of user choice as agreed upon by UX and Client. This shall be displayed directly below the project/showcase main details and above the screenshots gallery/slideshow.
 - One video walkthrough video shall be permitted per project, not per supported device/platform.
 - Videos (Dynamic Media) shall stored on a separate S3 Bucket from the Static Frontend Solution, with a bucket dedicated to Media Storage.
 - Videos shall be processed using in house AWS Media Services or a third-party service for transcoding and compressing video files received prior to storage in S3, outputting a standard format for consistency and optimised delivery to public users. Output format shall be webm for better compatibility with different browsers and devices. Estimated processing costs shall be approximately 50 cents per hour. Implementation Details shall be addressed by the Devs.
- **Download Options:**
 - Each available download/contact option (App Store, TestFlight, Website/Link to Showcase-Project, GitHub, Contact Us) shall be displayed as its own separate and independent button, rather than collapsed into a single dropdown menu section.
 1. **App Store Link and/or TestFlight Link (Minimum/Required, if available):** These links shall be provided as easier installation options compared with the manual download of the IPA file, for downloadable iOS app projects, and shall cover apps currently in beta testing (TestFlight) and/or apps that are fully live publicly via (App Store).
 2. **Link to Showcase/Project:** Specifically for projects that are not downloadable, like web-based projects.
 3. **GitHub Link (Optional):** shall be shown only if the project is open-sourced, and shall be used for version history.
 - *If a linked GitHub repository becomes private or is deleted later, the remaining App Store/TestFlight links shall remain sufficient to keep the page functional.*
 - **"How to Install" Guide:** A simplified guide shall be included to assist users with how to complete installations of downloadable projects or access non-downloadable projects.

- **Contact Us:** the page shall display a Contact Us option by default alongside the Download option on every project page. The HAPI shared email shall act as the intermediary and middle-man between visitors wishing to connect with the original developers, since the developers may have since graduated or otherwise moved on.
 - If a project has no download option available, the page shall display the Contact Us option in place of the downloads section, rather than disabling the or greying out the Download section.
 - This contact us option shall be a direct link to the shared HAPI email address, using the `mailto:` html attribute, allowing the user to send an email directly to the HAPI team. This shall not implement a custom pop-up form or messaging system.

Projects/Search Page:

- **Visual Rule #1:** The Projects/Search Results Page shall reuse the homepage's card grid layout, displaying each project's category in place of its short description.
- A **search bar** shall be positioned at the top of the page and shall match key terms against project Name and Description fields. Successful searches/filtering shall display a counter for "results matched" displaying the number of projects that matched the search criteria.
- **Standard Category** filtering shall be displayed on the left-hand side of the page. Category filtering options shall be revealed via clicking the menu button. A project shall now belong to multiple Standard Categories at once with a many-to-many relationship, and the Category filter shall allow multiple categories to be selected simultaneously, reflecting this updated relationship.
- **Showcase** filtering shall be displayed on the left-hand side of the page. Showcase filtering options shall be revealed via clicking the menu button. *Showcases shall optionally be further filtered using sub-categories as a Stretch Goal, from Program Names (AFP / Capstone / Apple Minor), to Year (2026/2027/etc.) and Semester (Semester 1 / Semester 2).*
- **Sort:** the site shall support sorting filtered/searched showcase projects by A–Z, Z–A, Newest, and Oldest. Newest/Oldest sorting shall be based on the project's associated Showcase publish date and not an individual project's creation date. *An optional "Popular" sort is being considered as a Stretch Goal.*
- **Pagination:** the site shall limit the number of projects displayed per page to 15, in a 5 row x 3 column grid, and narrowed down to 14 projects in a 7 row x 2 column grid when the filter side-menu is opened, and shall display numbered page buttons at the bottom of the page once that limit has been reached, which shall allow users to jump to a specific page of results.
- **Visual Rule #2:** The site shall visually indicate to a user which category/filter options are currently selected or deselected, so that the user see exactly what they have searched or filtered by. ***This issue was raised by Ace (HAPI Architect).***
- **Edge Case #1:** If a category is already selected and the user selects a different category, the newly selected category shall replace the previously selected one, instead of appending or adding to the list of selected categories for filtering.
- **Edge Case #2:** If a user clears/resets a search, only the search term shall be cleared, and any active sort or filtering selections shall remain unchanged.

Admin Dashboard Page:

- Dashboard for admins: admins shall be able to create additional admin accounts as needed, where authenticated admins shall each have their own individual login credentials.
- The Admin Dashboard shall be accessible only via a hidden, non-linked URL path: `/admin-login`, and shall not be discoverable through public navigation unless the url path is manually entered into the browser's address bar.
- Only authenticated admins shall be able to add, edit, publish/unpublish, approve/reject projects in the projects listing queue, with approval/rejection/pending remaining as Stretch Goals for the current sprint.
- Each project shall be assigned a Showcase category with a Program (AFP / Capstone / Apple Minor), Year (2026/2027/etc.) and Semester (Semester 1 / Semester 2). Standard categories such as Health/Workplace/Education/etc, shall not be fixed, and a project may now be assigned to multiple Standard Categories at once with a many-to-many relationship, which shall replace the previous one-to-one relationship with a single select dropdown now removed. Admins shall be able to select from an existing categories dropdown, or create new Categories directly in the Admin Dashboard, where even Featured Projects shall be selected, Showcases defined, and Admin accounts created.
- Each Showcase shall be individually definable and editable by admins via the Categories Section, with a Showcase Name, Year, Semester, Publish Date (Event Date), Description in a Markdown editor, Banner Image and assignable slugs, which shall be displayed on that Showcase's dedicated page. Each Showcases Publish Date, shall determine when its associated projects may be displayed on the public site. When a project linked to a Showcase with a Publish Date not yet reached, it shall be treated as Unpublished regardless if the project details are ready and approved or not.
- **External Dependency — Student Project Submission Details via HAPI Email:**
 - *Students shall be expected to email their project submission details to the shared HAPI email address, as an automated student project submission portal/dashboard is currently out of scope.*
 - *Admins shall be expected to coordinate with these students to ensure the following details are included before a project can be reviewed/approved and manually added to the site: Icon, Name, Subtitle (Short Description), Long Description, Developer Name/s, Category, Screenshots/Images (minium of 4 per supported device), Optional Video Walkthrough Link, Supported Platforms (iPhone/iPad, iPad, macOS MacBooks, and/or Vision Pro), and relevant download/source link/s, such as App Store, TestFlight, and/or GitHub repo if open-sourced.*
- **Adding Projects:** An "Add Project" button shall be displayed on the top-right corner of the Admin Dashboard. Clicking it shall navigate the admin to a dedicated Add Project page, which shall track and save changes live as the admin enters details, for the admin to manually enter a new project's details. This page shall also support uploading screenshots per device gallery, a single video walkthrough, an app icon, a Hero Banner Image, assigning multiple Standard Categories, and a slug field which shall validate against already taken slugs by other projects prevent reuse, and a include markdown editor for the project's long description.
- **Featured Projects:** admins shall select projects as Featured via a heart-button option, and rank them via a drag-and-drop feature and/or up/down arrow buttons provided for keyboard accessibility as an alternative to drag-and-drop ranking.
- **Layout:** The Admin Dashboard shall use a similar layout to the Projects/Search Results Page, except displaying listings as a horizontal list of cards, one project per row, instead of 2-3 or more columns that

the Projects/Search page displays depending on screen size. Each row shall show the project's icon, name, short description, public visibility status (Published/Unpublished). *The editorial review status (Pending/Approved/Rejected) shall remain a Stretch Goal to complement public visibility with a review stage for the Student Submission Portal Stretch Goal if implemented.* This design choice shall separate the admin and user interfaces visually to avoid any potential confusion between the two.

- Review Statuses (Approved/Rejected/Pending) shall represent the project's editorial review stage, and tracked separately from whether a Project is Published or Unpublished for public visibility/accessibility. This separation shall allow an Approved project to be temporarily Unpublished for quick content updates, and shall take into consideration the possibility of a Self-Serve Student Portal to fully leverage the Pending Queue and review projects in an efficient manner.
- **Pending** shall indicate the project entry is incomplete or awaiting further details from the student before it's ready for review and approval.
- **Rejected** shall indicate a completed project entry that the admin has chosen not to make public. *Taking down a previously public project without deleting it shall be handled via the Published/Unpublished status instead.*
- **Approved** shall indicate a completed project entry that has passed review, however, the project shall still requires to be set Published for it to be visible on the public site.
- **Additional Actions:** Each row representing a project shall display an "Action" button on the right-hand side, which shall show either an "Edit" and "Delete" button
 - Clicking the "**Edit**" button shall navigate the admin to a dedicated Edit project page, which shall match the Add Project page, while tracking live changes for editing the current project's details.
 - Clicking the "**Delete**" button shall open a pop-up form for confirming the deletion of the project from the database.
 - *Stretch Goal - Recycle Bin: shall send deleted projects to a recycle bin, which shall allow an admin to recover/restore an accidentally deleted project from the recycle bin, before a final permanent deletion step is taken. This shall prevent the immediate deletion of a project, which shall further complement the confirmation "Delete" button.*
- **Sort/Pagination:** The Admin Dashboard page shall share the same sort/pagination behaviour and rules as the Projects/Search page. Sort Labels shall be displayed in the following order: A–Z, Z–A, Newest, Oldest.
- **Filtering:** the Admin Dashboard page shall provide three separate filter groups, one for Status (Published/Unpublished), one for Showcase Categories (AFP / Capstone / Apple Minor), and one for Dynamic Categories (AFP, Capstone, Apple Minor).

- Within each group, only one option shall be selectable at a time, similar to the Projects/Search page, besides the Standard Category filtering, which shall be more dynamic allowing for multiple categories to be selectable at the same time, and shall reflect the revised many-to-many relationship between projects and categories similar to a tagging feature.
- The three groups shall be combinable with each other at the same time, so that users can filter by both status, showcase and category all at the same time. This shall give admins greater control and refinement over what they're viewing.
- **Edge Case #3:** If an admin changes the selection within one filter group, then only that group's filter option shall update, the other filter group's current selection shall remain unaffected.
- When a project's status is set to rejected, the admin shall be required to enter a short feedback/reason for the rejection, which shall be stored as part of the project's record.
- **Stretch Goal - Audit Logging:** *The system shall track which admin made each change to a project record, whether adding/editing/deleting or updating the project's status, for accountability and traceability, complimenting the addition of the multi-admin accounts support feature. If implemented, this log shall be viewable by admins within the Admin Dashboard and within each Edit Project page.*

Stretch Goal #1 - Web Wrapper Application:

- *The system shall support the delivery of a lightweight wrapper packaging the existing website as an installable, app-like experience, whether for iPad or iPhone, rather than a complete rebuilding of the site.*
- *This additional consideration is not a core deliverable of this project, but shall be a "Stretch Goal" brought up by the Client as declared in the originally signed Client Project Proposal document.*
- The dev team proposed the Progressive Web App (PWA) approach as the most suitable implementation route for this wrapper, offering an installable, app-like experience without a separate native build. This direction was agreed upon as a deliverable by the Client once the MVP build sprint stage is complete.

Stretch Goal #2 - Self-Serve Student/Developer Submission Portal (in Development):

- Students shall be able to create an account via a sign-up/login dashboard and submit their project applications directly. Student emails shall be validated against a front-end JS email validation rule - ending with `@student.rmit.edu.au` and verified via a link sent to their email. Student submissions/applications shall be added to a waitlist queue table under a pending status until their project is reviewed and approved/rejected.
- **Justification:** This approach was considered more secure compared to the alternative with a fully public, unauthenticated submission form, which would introduce the risk of spam or fake submissions. However, since this also requires individual developer/student accounts on the site, the Client has decided to forgo this approach for now, until the main scope of this project has been completed, and time allows at the end of the sprint to re-evaluate as part of the final deliverable.

Future Consideration #2 - Project History/Inspiration Section:

- This section shall allow the story behind a project to be shared, such as what inspired it, the development journey, challenges faced along the way, the goals/vision for the project and a link/association to the real world campus location where the project was created, with photos of the space and experience. Similar to all other project details, the contents of this section shall be collected via project submission applications sent to the shared HAPI email address, then manually added to the site. This section shall be displayed at the bottom of the project page if implemented.
-

2. Non-Functional (Business) Requirements/Rules

Accessibility:

- The site shall meet WCAG (W3C) basic guidelines, including Alternate text for Images, Keyboard Navigation, Reasonable Colour Contrast, Text Resizing/Scaling Support.
 - Scoped as part of Sprint 2 build/development stage, not Sprint 1.
 - Colour contrast has already been considered during the UX Design stage by Max Thum, ahead of the full accessibility implementation stage dedicated to Sprint 2.

Performance & Traffic:

- **Assumption:** While the site shall be Public-facing, traffic shall be expected to remain low overall, with short predictable spikes during HAPI showcase events (via QR code links to the project pages).
- The system shall support at least 10-12 projects at launch, scaling incrementally as more projects are approved and added over time.
 - *Stretch Goal - Search Engine Optimisation (SEO): SEO shall be a "Stretch Goal" deliverable for this project, and shall be implemented once the core site functionality meets the project's base scope. If SEO is implemented, it shall support individual projects and the overall HAPI Showcase website receive additional traffic and increase visibility.*
- **Key Performance Metrics That Shall be Tracked Include:** site traffic, API response times, Lambda cold start times, and page load times. An analytics service shall be considered for assisting gathering these metrics more easily, and shall serve as a benchmark for the HAPI team after handover is completed post project closure. Two options shall be currently under consideration: Umami and Google Analytics, with a final decision pending.

User Acceptance Testing (UAT):

- Formal end-to-end UAT scenarios covering all functional requirements and identified edge cases shall be documented as a separate deliverable, covering both General/Public Access and Admin Access perspectives, with accessibility, backend and infrastructure concerns excluded from the scope. This shall serve as a working checklist against the current Requirements Specification Document, and shall be iteratively refined over the next two sprints following feedback and review from the team and client.

User Experience Testing:

- In addition to functional User Acceptance Testing (UAT), a subjective and experience-based testing shall be conducted via a focus group of around 5 participants, organised by the Client Michael of internal members from the HAPI group. This shall complement the UAT checklist-based UAT Testing covered separately for functional requirements, focusing instead on overall user experiences and satisfaction acting as guide for the UX Designer Max Thum. Results shall also serve as a baseline for the HAPI team to reference when considering future improvements after handover is completed post project closure.

Availability:

- **The Website shall be reachable via a yet to be confirmed address:**
 - *subdomain-based URL structure:* `hapi.rmit.edu.au`
 - *path-based routing URL:* `rmit.edu.au/hapi`
- **Fallback:** if no hosting platform decision can be reached in time, a Packaged Site including a Database export delivered via the GitHub repository shall be an acceptable substitute deliverable.
- **Redundancy & High Availability:** will be covered in the Backend Architecture section.

Budget:

- No fixed budget limit has been set by the client beyond a general guideline of cost-minimisation and being in agreement with the dev teams estimation for ongoing AWS costs as approximately \$25/month, which shall be a reasonable starting point for the project. The client has informally suggested a rough figure of \$600/year, which shall still be considered, but requires confirmation from HAPI's own upper management. The AWS RDS database shall be acknowledged as the most costly component of the project. Video transcoding/compression processing is estimated at approximately \$0.50/hour.

Security

- Authentication shall be scoped to admin access only, with no individual author/user accounts included as part of this project's current scope.
- All data in transit, especially admin login credentials, shall be secured via TLS/SSL encryption.
 - *AWS Certificate Manager (ACM): comes free with API Gateway.*
- All data at rest, including admin credentials and any future developer contact details, shall be encrypted natively by the chosen database solution, whether AWS RDS with Postgresql or Supabase.
 - **Stretch Goal - Two Factor Authentication (2FA):** *2FA shall be a "Stretch Goal" deliverable for this project, and shall be implemented once the core site functionality meets the project's base scope. If 2FA is implemented, it shall enhance security by requiring an additional layer of authentication to access the admin dashboard.*

Branding

- The Visual Direction & Design shall blend an Apple App Store aesthetic with RMIT/HAPI identity and branding, incorporating the HAPI logo.

Maintainability

- Ace (HAPI Platform Architect) shall be the designated contact for handover and post-project maintenance.
- Handover and maintenance documentation shall be published in the project's Github repository under the `docs/` directory.

3. Backend Architecture Design — Proposal & Ideas:

This shall detail a list of possible solutions for the backend architecture of the HAPI Showcase. While nothing is finalised here, these shall be working ideas for our Dev to confirm and refine once we are authorised for access to RMIT's RACE/AWS infrastructure.

- **Versioning:** GitHub shall be our main solution, with each project's version tracking changes, updates and releases.
- **Hosting:**
 - **Main:** Shall use RACE (AWS), now approved by RMIT.
 - The architecture solution shall leverage a serverless platform using AWS Lambda functions and API Gateway for backend functionality, a static website hosted on S3 for the front-end, and a database hosted on AWS RDS.
 - **Alternative/Backup Option:** Vercel shall remain a backup alternative, however RACE approval has already been granted.
 - **MVP Delivery & Handover Redeployment:** Terraform shall be utilised regardless of any blockers or delays for implementing Infrastructure as Code provisioning, while the MPV site shall leverage a free-tier AWS account with Supabase as the database solution, and the project's Github repository as the source code repository, which shall allow the HAPI team to fork and maintain the project themselves post handover.
- **Storage:** Shall use S3 Bucket (AWS) for Media Storage
 - **Screenshots/Icons (Static Media):** shall be stored on the same S3 Bucket as the Static Frontend Solution. Screenshots shall be organised per supported device (iPhone/iPod, iPad, macOS, Vision Pro, Apple Watch, Apple TV).
 - **Video (Dynamic Media):** Shall be stored in a separate S3 Bucket for dedicated video storage.
 - Videos shall be processed using in house AWS Media Services or a third-party service for transcoding and compressing video files received prior to storage in S3, outputting a standard format for consistency and optimised delivery to public users. Output format shall be webm, for better compatibility with different browsers and devices. Estimated processing costs shall be approximately 50 cents per hour. Implementation Details shall be addressed by the Devs.
 - S3 Video Storage shall be the confirmed solution for video storage, with YouTube embedding effectively removed from scope altogether.
- **Data Store (Database):**

- **Main:** Shall use Postgresql via AWS RDS, and a database schema shall be prepared based on the required attributes of each HAPI showcase project.
- **Alternative/Backup Option:** Shall use Supabase, if RDS turns out too costly or otherwise not viable.
- **Project Records Required Attributes:** Each project shall be represented as a database record containing the following attributes:
 - *ProjectID, Icon, Name, Subtitle (Short Description), Description (Long Description), Developers (Names), StandardCategories (many-to-many relationship), ShowcaseCategory_Program, ShowcaseCategory_Year, ShowcaseCategory_Semester, DateCreated (Optional for information purposes only, for project visibility shall be determined by its associated showcase publish date instead), PlatformVersion, SupportedDevices (iPhone/iPod, iPad, macOS, Vision Pro, Apple Watch, Apple TV), AppStore_Link, TestFlight_Link, ProjectLink, GitHubLink, HeroBannerImage, VideoWalkthrough (Optional, one per project), ScreenshotsFolder (per-device: ScreenshotsiPhone, ScreenshotsiPad, ScreenshotsMacOS, ScreenshotsVisionPro), PublishedStatus (Published/Unpublished), Slug (unique and managed by admins), ReviewStatus (Approved/Rejected/Pending - *Stretch Goal*), RejectionFeedback (or RejectionReason - *Stretch Goal*), FeaturedProject (Boolean True/False for Drag and Drop after Choosing Project as Featured Showcase on Homepage).*
 - **Showcases:** shall be represented as a separate table with a ShowcaseID, Name, Program, Year, Semester, Description, BannerImage, PublishDate and Slugs.
 - **Note:** *These attributes cover the necessary details for a project, per a specification requirements perspective, therefore, data types, rules and structure shall be determined by the development team.*
 - **Stretch Goal - Multi-Admin Accounts Audit Logging & Recycle Attributes:** *CreatedByAdminID, LastModifiedByAdminID, InRecycleBin (Boolean: True/False).*
- **Project Application Approval Status:** Projects shall have an assignable approval-status field/attribute where only approved projects are visible on the public site. The ReviewStatus attribute (Approved/Rejected/Pending - *Stretch Goal*) shall represent the project's editorial review stage only, while the separate PublishedStatus attribute (Published/Unpublished) shall independently control whether a project is visible on the public site. Default values shall be:
 - **Published:** Shall indicate the project is currently visible on the public site.
 - **Unpublished:** Shall indicate the project is hidden from the public site, regardless of whether it has been approved or not.

 - **Pending:** Shall be currently under review.
 - **Approved:** Shall be ready for public display on the site.
 - **Rejected:** Shall include a stored feedback/reason attribute, which is provided by the admin at the time of the rejection of the individual project.
- **Project Application Actions:** Projects shall be manipulatable by the admin, where new projects can be added, existing projects can be edited or permanently deleted.

- **Add/Edit:** Shall be standard create and update actions for new and existing projects as records in the database. Add shall create a new entry via the Admin Dashboard's dedicated "Add Project" page, and "Edit" shall update an existing entry via a similar dedicated "Edit Project" page, pre-filled with the current project's details and tracking changes live as they're made. Both pages shall support uploading screenshots for the project's gallery per supported device (iPhone/iPod, iPad, macOS, Vision Pro, Apple Watch, Apple TV), a single video walkthrough file, a slug field for the project's unique URL, the project's icon image, a Hero Banner Image, assigning multiple Standard Categories, and include a markdown editor for the project's description.
- **Delete:** Shall be a distinctly permanent action from "Rejected", which shall remove the project entirely, while "Rejected" retains the record but hides it from the public view. Deleting a project shall also delete its associated screenshot images, project icon, Hero Banner Image, and video walkthrough.
- **Stretch Goal - Recycle Bin:** *shall send deleted projects to a recycle bin, which shall allow an admin to recover/restore an accidentally deleted project from the recycle bin, before a final permanent deletion step is taken. This shall prevent the immediate deletion of a project, which shall further complement the confirmation "Delete" button.*
- **Contact US:** Shall use the HAPI email address for direct messaging, which shall leverage the `mailto:` html attribute link, allowing the user to send an email directly to the HAPI team.
 - **Note:** *Although the Contact Us option shall visually replace the Download button on the frontend when a project has no downloadable link, this shall only be a display isolated feature/behaviour, while the database schema shall still retain the download related fields for that project record, but empty/null.***
- **Search/Filtering:** Shall use a simple and quick server-side keyword search for partial matches on a Project Name and/or it's Description, where an exact match is not required, alongside Category Filtering. Server-side search was confirmed by Dev team, which shall offer easier backend integration, no need to load the full project dataset to the front-end, and better scalability as the number of projects grows, with only a minor trade-off of added network/query latency per search. Successful searches/filtering shall return a counter for the number of results matched. Category filtering options shall be revealed via hover to the left or click menu button on the frontend. This search functionality shall be accessible from both the homepage and the dedicated Projects/Search page.

4. Frontend Interface Design — Proposal & Ideas:

Nothing shall be considered finalised here, as these shall be working ideas for our UX to confirm and refine into a design that reflects a clearer visual representation of what the HAPI Showcase website shall look like.

- **Overall Layout:** Shall blend an Apple App Store aesthetic fused with RMIT/HAPI branding, clean grid layouts, card-based browsing and clear imagery for individual project pages. The design shall be responsive to different screen sizes, accounting for iPad, iPhone and web/desktop.
- **Homepage:** Shall display a Welcome area, a search bar, and a Featured Projects Section. The Featured Projects section shall spotlight two projects to stand out with their Hero Banner Image, and all remaining featured projects underneath and shall be displayed using the standard Search/Project cards, with no currently fixed limit on the number of projects that shall be marked as Featured. On every page refresh, different projects from the Featured Projects section shall be randomly selected and displayed as spotlight projects. This revised design choice shall be more “democratically” aligned, addressing client concerns around favouritism amongst student projects. Key Project Details shall be focused, including icon, name, short description and label for Category (AFP, Capstone, Apple Minor), where clicking any project card shall direct users to the individual project, and clicking the “View More” shall direct users to the project/search page. The homepage shall also include a search bar, allowing users to search without needing to navigate to the Projects/Search page. Featured Projects shall be manually selected by admins via the Admin Dashboard’s “Featured Projects” section, using heart-button for selection, and ranking via drag-and-drop or up/down arrow buttons as a keyboard accessible alternative.
- **Individual Project Pages:** Shall include icon, name, subtitle (short summary), developers, Date Created (*Optional for information purposes only, for project visibility shall be determined by its associated showcase publish date instead*), Showcase category: Program/Year/Semester (*with sub-categories*), standard category, Supported Devices (iPhone/iPod, iPad, macOS MacBooks, Vision Pro, Apple Watch, Apple TV), long description, screenshots (shall display a single combined carousel across all supported devices by default as, and support narrowing down to specific supported devices from there, with a minimum of 4 per supported device, shown in carousel or grid format), optional video walkthrough that is user-initiated but muted by default and positioned above the screenshot gallery, download/source link(s) (App Store and/or TestFlight, Link to Non-Downloadable Projects, GitHub for open-sourced projects), and a contact us button with a `mailto:` link to the HAPI email address shown by default which also replaces the download option if the project is not downloadable. *“How to Install” Guide: shall be provided to assist users with how to complete installations of downloadable projects.*
- **Project Search Page:** Shall reuse the same card grid as the homepage for projects, with a search bar at the top of the page, filtering categories on left side of page, , Sort and Pagination controls, a counter for “results matched” on successful search/filtering, a visual indicator showing which filters are currently selected/deselected, and a no-results message when projects don’t match the user search terms. Sort options shall be clearly labelled and shall include: A–Z, Z–A, Newest, Oldest. *_An optional “Popular” sort is being considered as a Stretch Goal.*
- **Admin Dashboard:** Shall retain each project’s icon, name and subtitle (short description), but displayed it shall be displayed as a horizontal list of cards, with one project per row, rather than 2-3 or more columns that the Projects/Search page displays depending on screen size. It shall also include an “Add Project” button in the top-right corner of the page, which shall navigate the admin to a dedicated Add Project page for manually entering new projects, as well as an *Stretch Goal Editorial “Review Status”, either Pending/Approved/Rejected with a stored feedback/reason attribute for rejected project applications*, a separate independent Published/Unpublished visibility status, and an

“Edit” button per row/project, which shall shall navigate the admin to a dedicated Edit Project page for for editing the project’s details or deleting the project entirely after a confirmation popup appears to verify the action, ***or, if the Recycle Bin stretch goal is implemented, then it shall move the project directly to the recycle bin first.*** Review Status changes between Pending/Approved/Rejected and Visibility Status changes between Published/Unpublished shall be done via the dedicated Edit page.

- The Admin Dashboard shall be presented with a horizontal top menu bar, first with the “Projects” section for searching/filtering projects, second the “Featured Projects” section for assigning and ranking the featured projects for display on the homepage, third the “Showcase” section for adding/editing the Showcase categories, fourth the “Standard Categories” section for adding/editing the Standard categories, fifth the “Admin Users” section for managing admin user accounts, and sixth the “Analytics” section for viewing analytics data.
- The filtering categories within the Admin Dashboard’s “Projects” section shall be displayed on the left-hand side of the page, which shall allow for combining of the Status Group and Category group for further narrowing of results. The exact filtering rules allowed in the Admin Dashboard shall be declared in the Functional Specifications of the Admin Dashboard Page above. Furthermore, the Admin Dashboard shall share the same sort/pagination behaviour and rules as the Projects/Search page. This design choice shall separate the admin and user interfaces visually to avoid any potential confusion between the two. The dashboard shall also support.
- The “Featured Projects” section shall support selection of Featured projects via a heart-button option, and ranking via a drag-and-drop feature and/or up/down arrow buttons provided for keyboard accessibility as an alternative to drag-and-drop ranking.

Handoff to UX Notes:

- As the specifications requirements have evolved over the past few iterations, additional changes requiring the UX to be aware of shall be listed here. These include:
 - Preparation for the UX Focus Group Survey in Week 7, Sprint 3 Week 1 now that the Survey Questions have been reviewed and finalised.

Deliverable:

- This Requirements Specification document shall be delivered as a markdown file `.md`, named `requirements-specification.md` and committed to the project’s `docs/` directory.
- The document shall be shared as a deliverable with the UX for refinements and feedback, then the Dev for implementation.